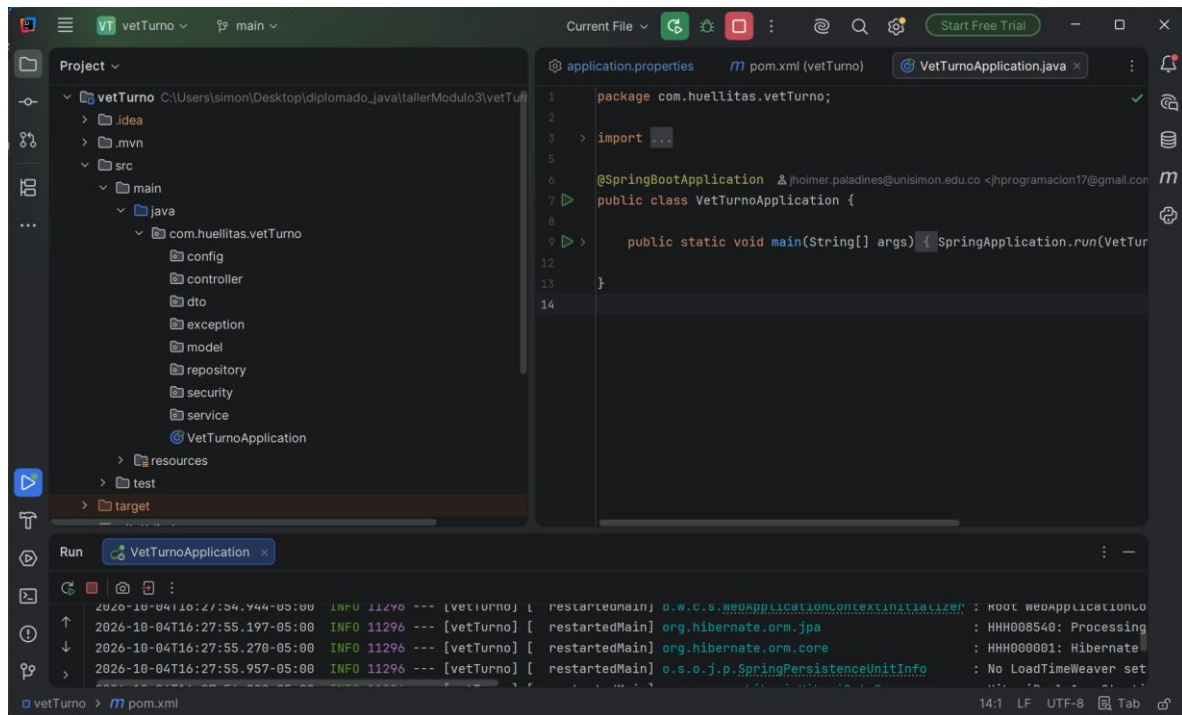


Captura del inicio correcto del servidor y del árbol de paquetes:



Enlace al commit inicial:

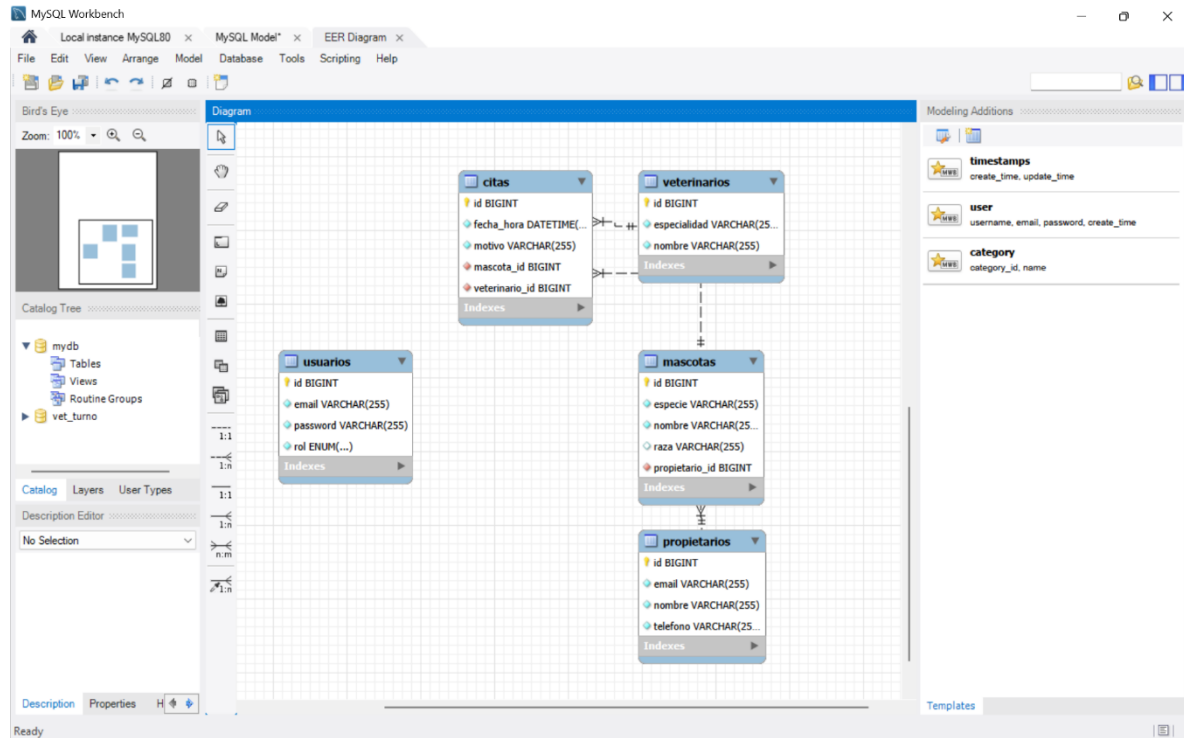
<https://github.com/jhprogramacion17-source/vetTurno/commit/e9cddd10fec226f1797af895ca0e38a42ca76ee5>

Explicación breve de la diferencia entre pom.xml y la clase principal:

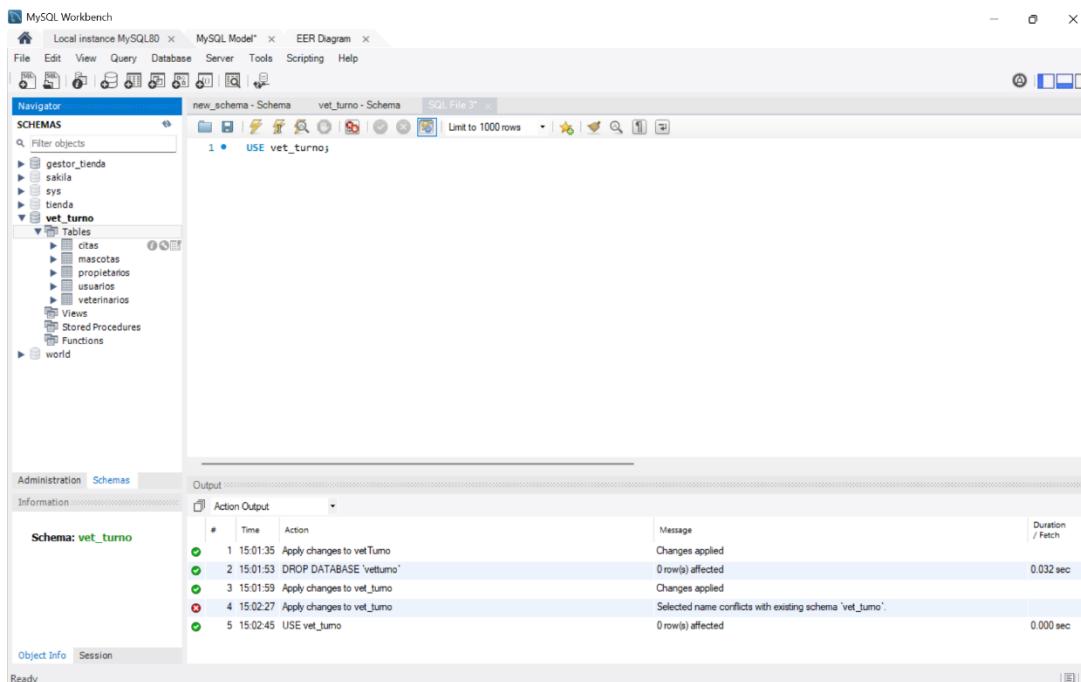
La clase principal es donde la app se inicializará siempre, cada vez que queramos ejecutarla siempre será desde ahí, en cambio el archivo pom.xml controlado por Maven es donde se guardan, instalan y administran todas las dependencias o librerías que se usan en la app.

PARTE 2

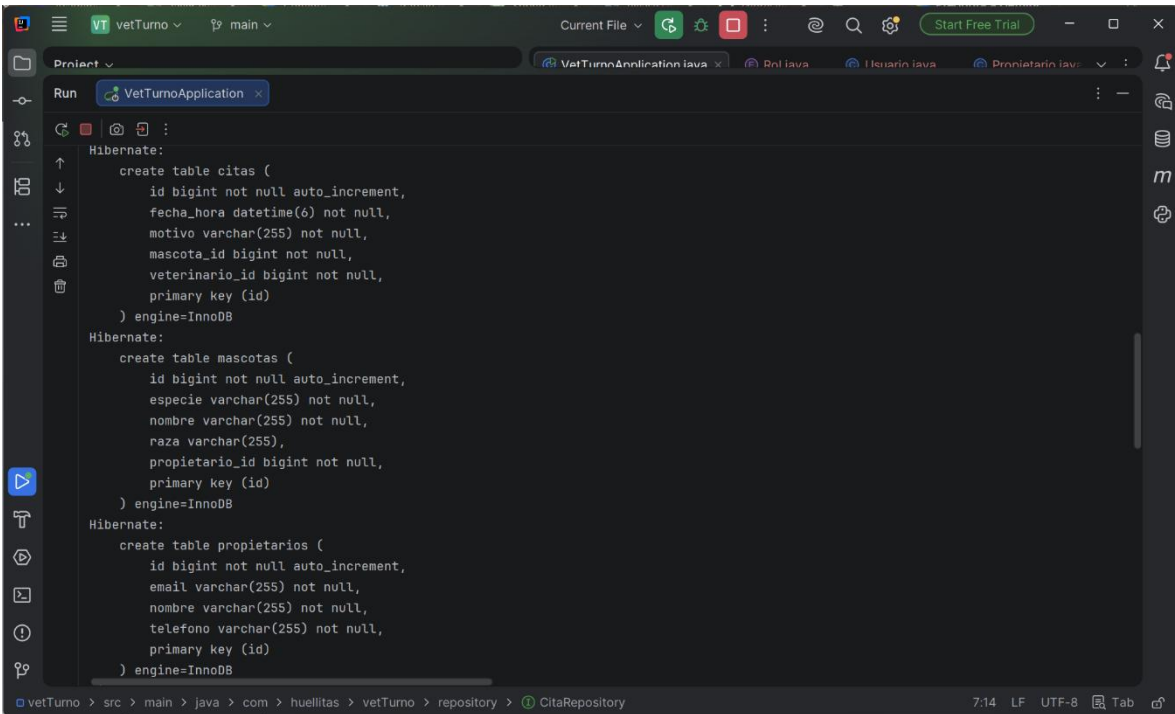
Diagrama sencillo del modelo o captura del esquema relacional.



Captura del SQL generado al iniciar y de las tablas en Workbench



Captura del SQL generado al iniciar y de las tablas en Workbench



```
Run VetTurnoApplication x
Hibernate:
create table citas (
  id bigint not null auto_increment,
  fecha_hora datetime(6) not null,
  motivo varchar(255) not null,
  mascota_id bigint not null,
  veterinario_id bigint not null,
  primary key (id)
) engine=InnoDB
Hibernate:
create table mascotas (
  id bigint not null auto_increment,
  especie varchar(255) not null,
  nombre varchar(255) not null,
  raza varchar(255),
  propietario_id bigint not null,
  primary key (id)
) engine=InnoDB
Hibernate:
create table propietarios (
  id bigint not null auto_increment,
  email varchar(255) not null,
  nombre varchar(255) not null,
  telefono varchar(255) not null,
  primary key (id)
) engine=InnoDB
```

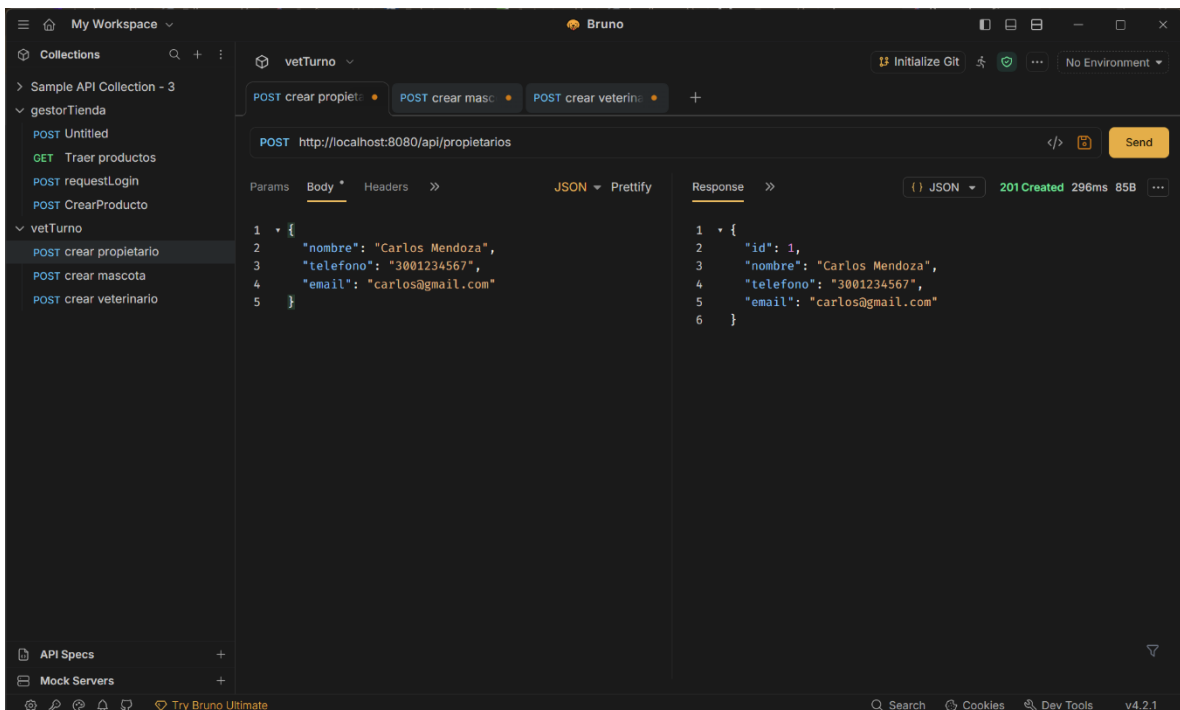
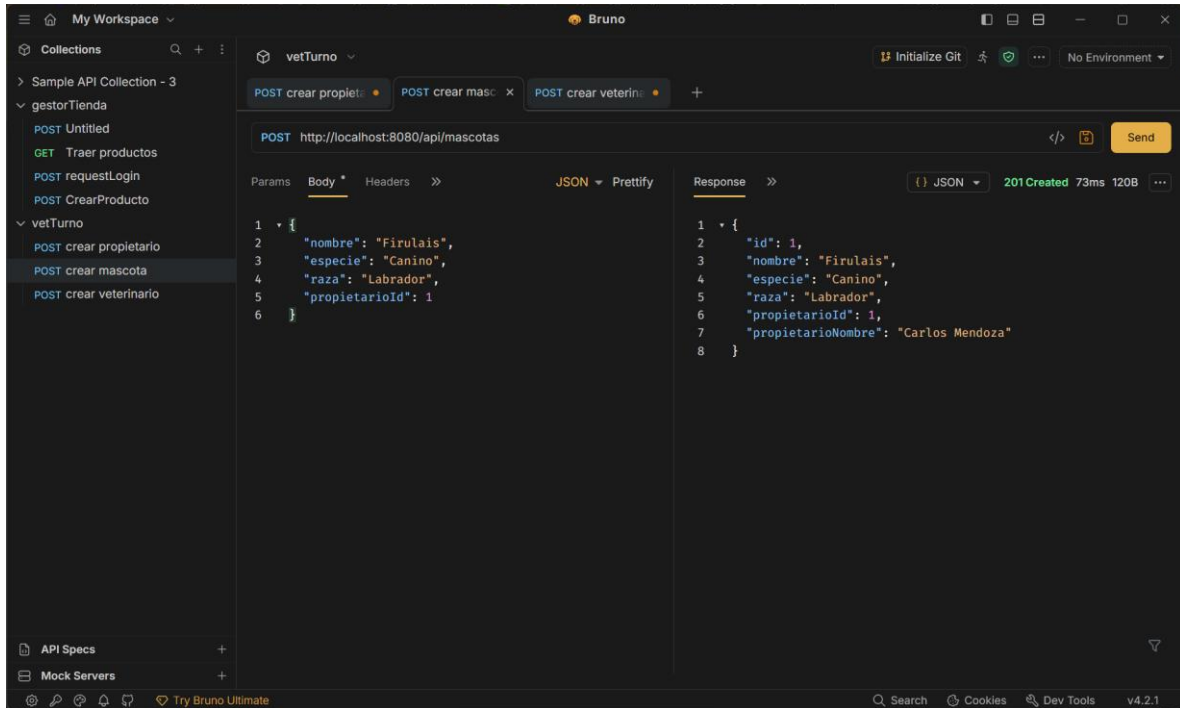
vetTurno > src > main > java > com > huellitas > vetTurno > repository > CitaRepository 7:14 LF UTF-8 Tab

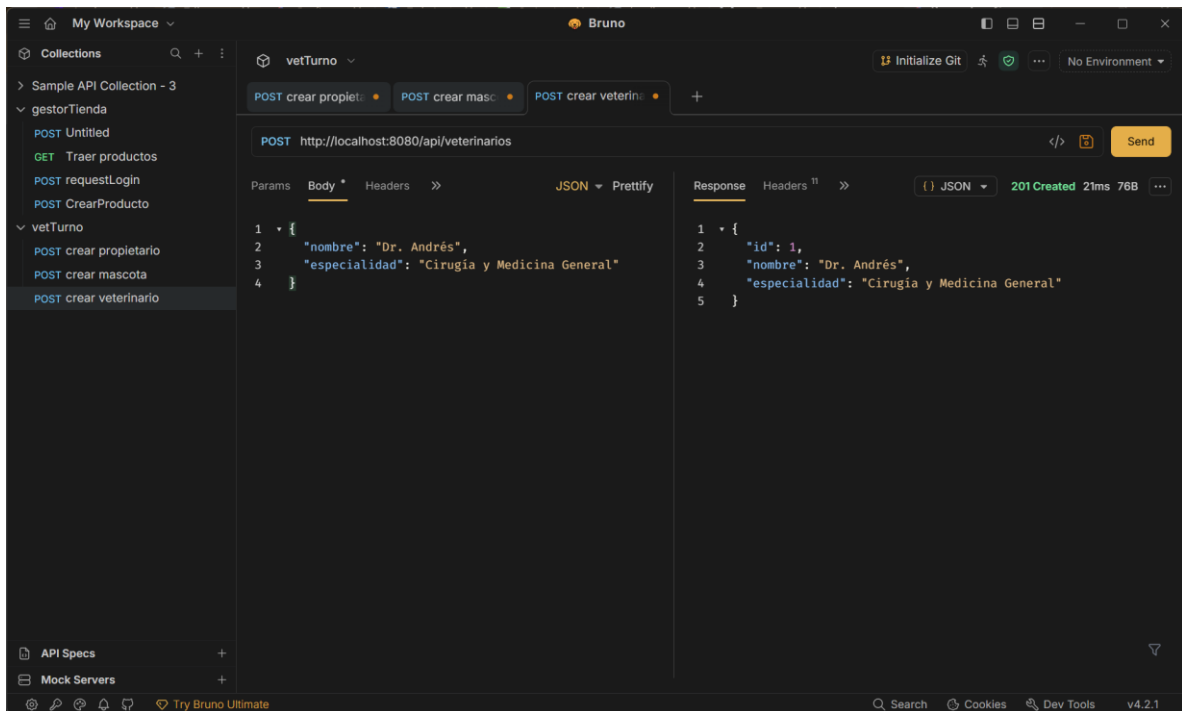
Explicación de la diferencia entre JPA, Hibernate y JpaRepository:

JPA es la especificación estándar que define las reglas de mapeo mediante anotaciones como `@Entity`, `@Table`, `@Id` y `@ManyToOne`, además de interfaces como `EntityManager`. Con base en estas indicaciones, **Hibernate** actúa como el motor ORM que interpreta el mapeo y ejecuta internamente las sentencias SQL contra la base de datos. Por último, **JpaRepository** (de Spring Data) es la abstracción que automatiza las operaciones CRUD y las consultas derivadas, permitiéndonos trabajar sin tener que gestionar el `EntityManager` de forma manual.

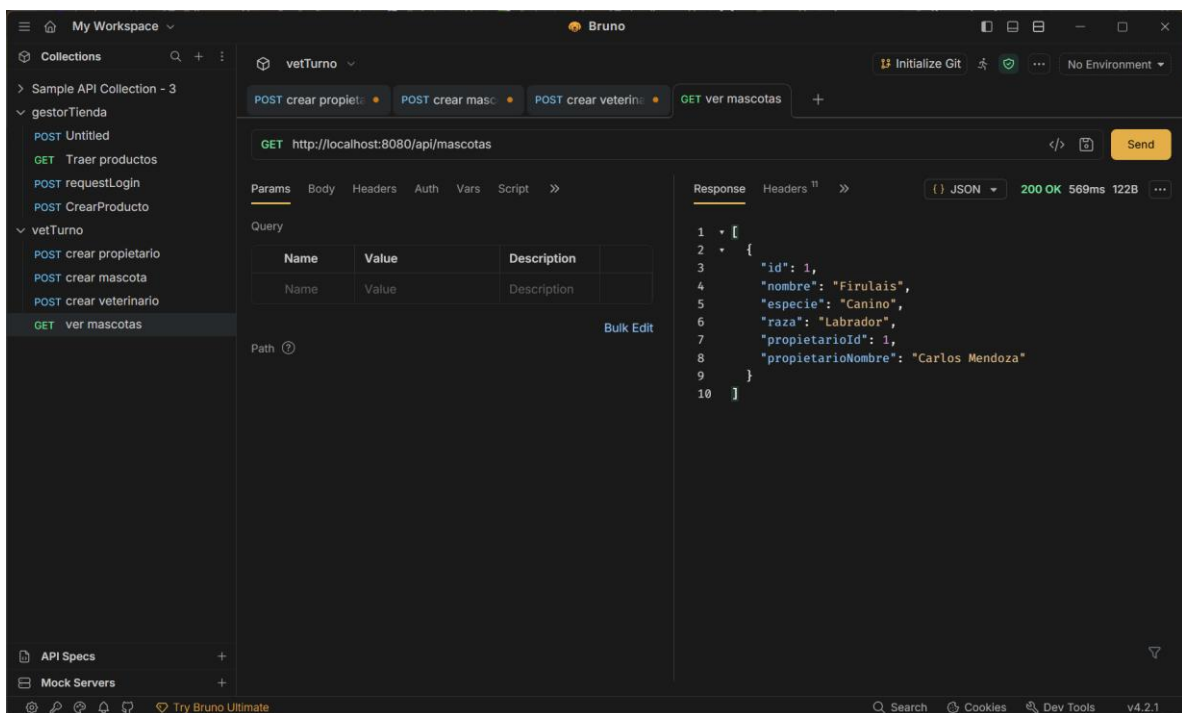
PARTE 3

Peticiones de creación y consulta de las tres entidades:



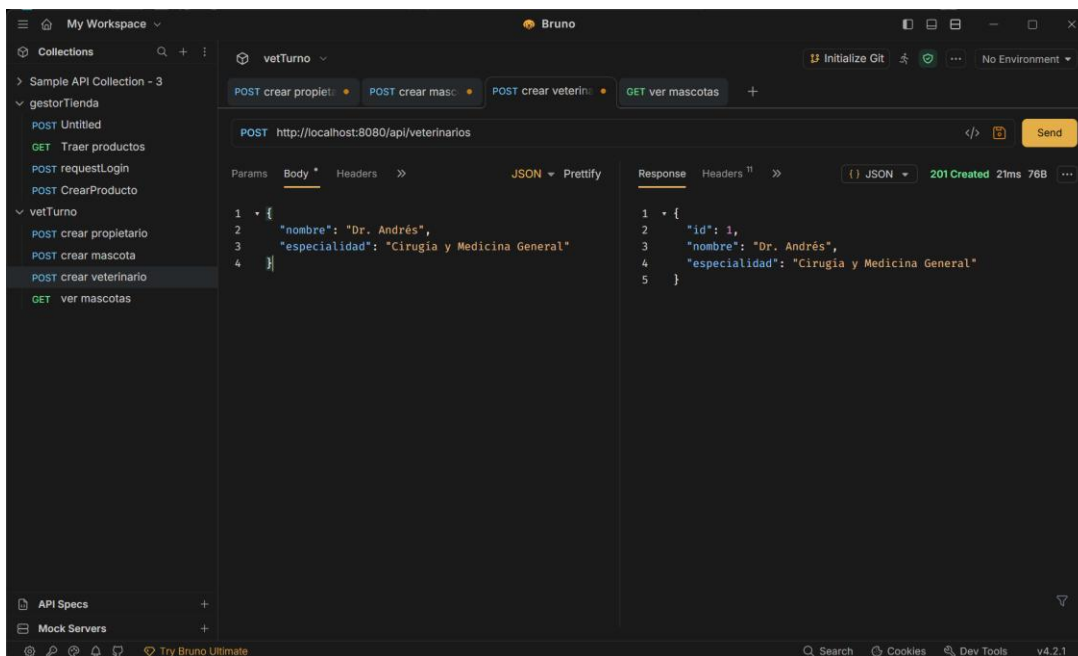


Respuesta de una mascota con información breve de su propietario:

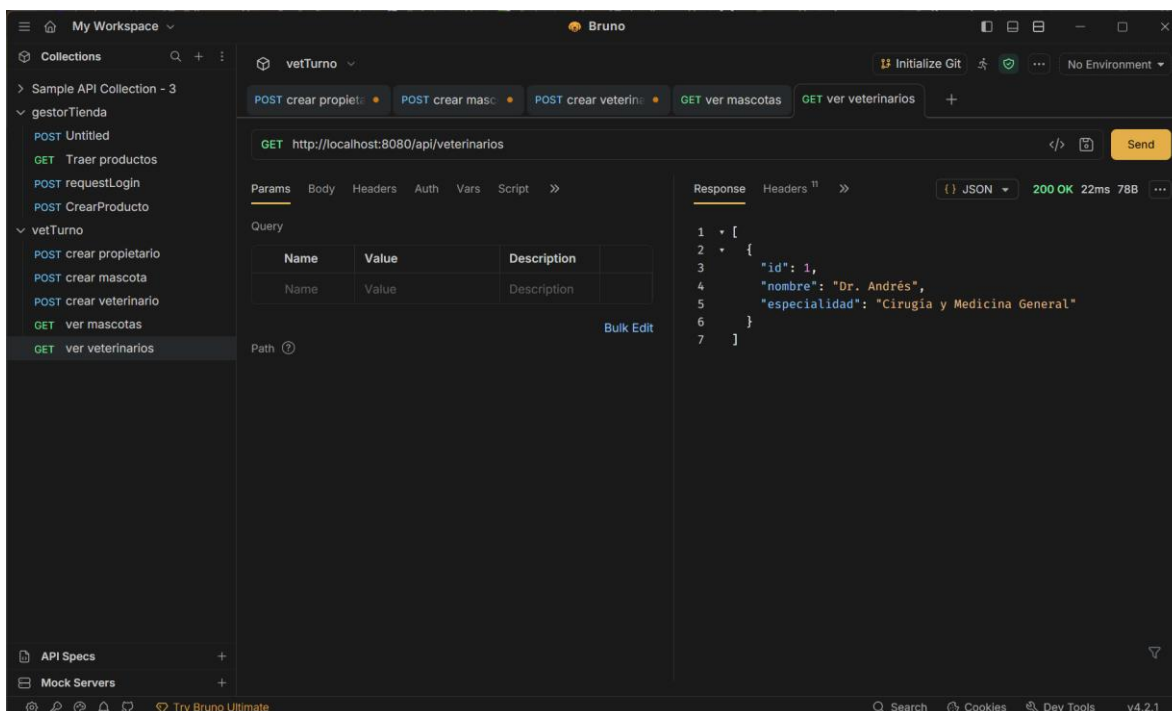


Demostración de los datos antes y después de reiniciar el servidor:

ANTES:

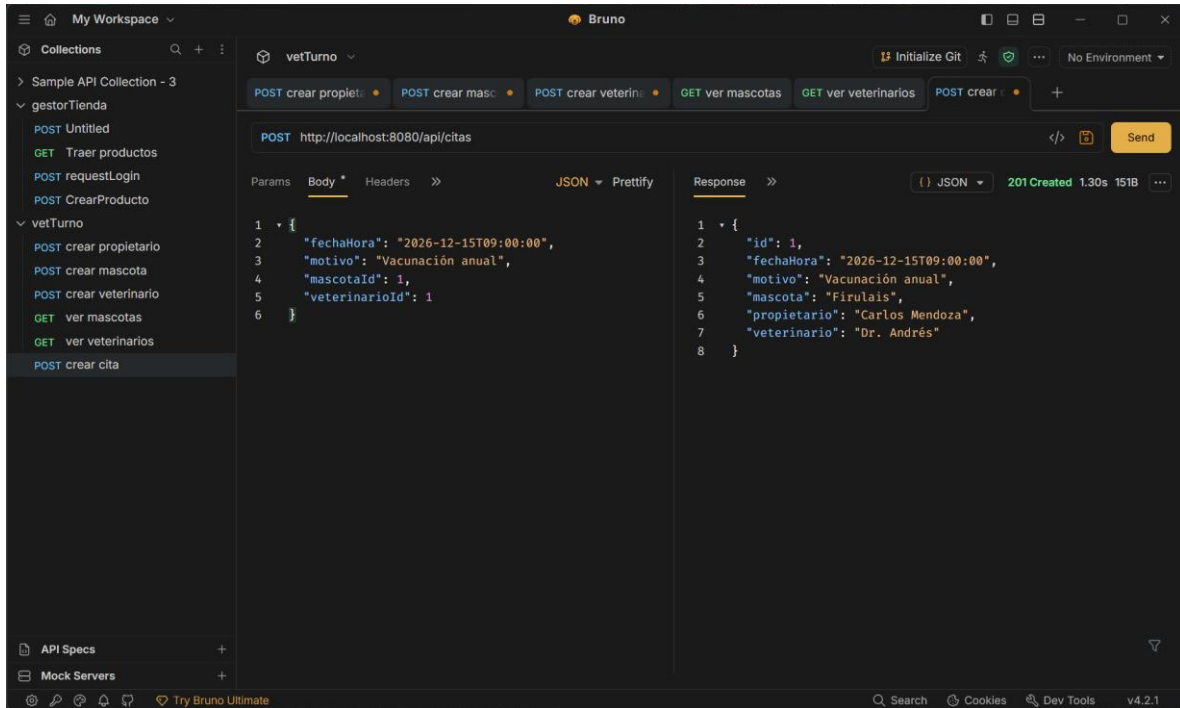


DESPUES:

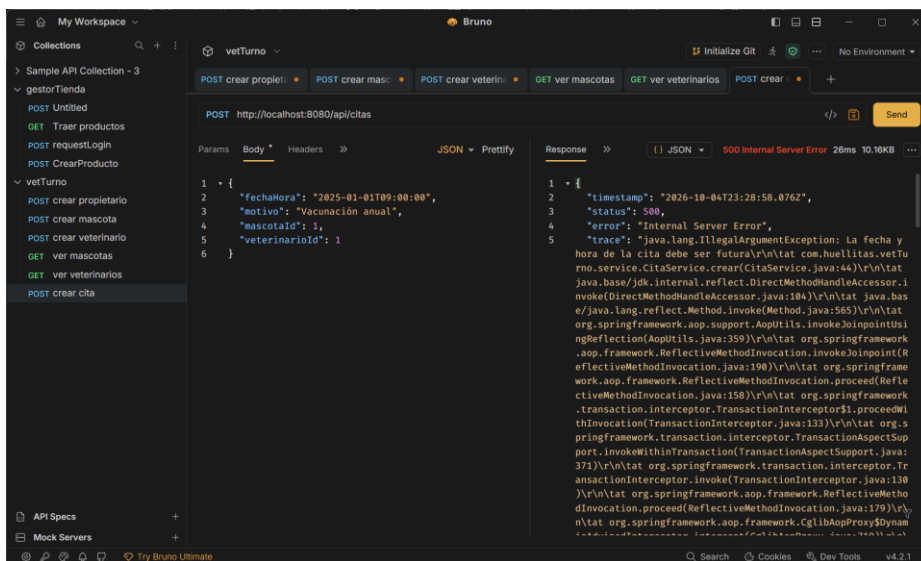


PARTE 4:

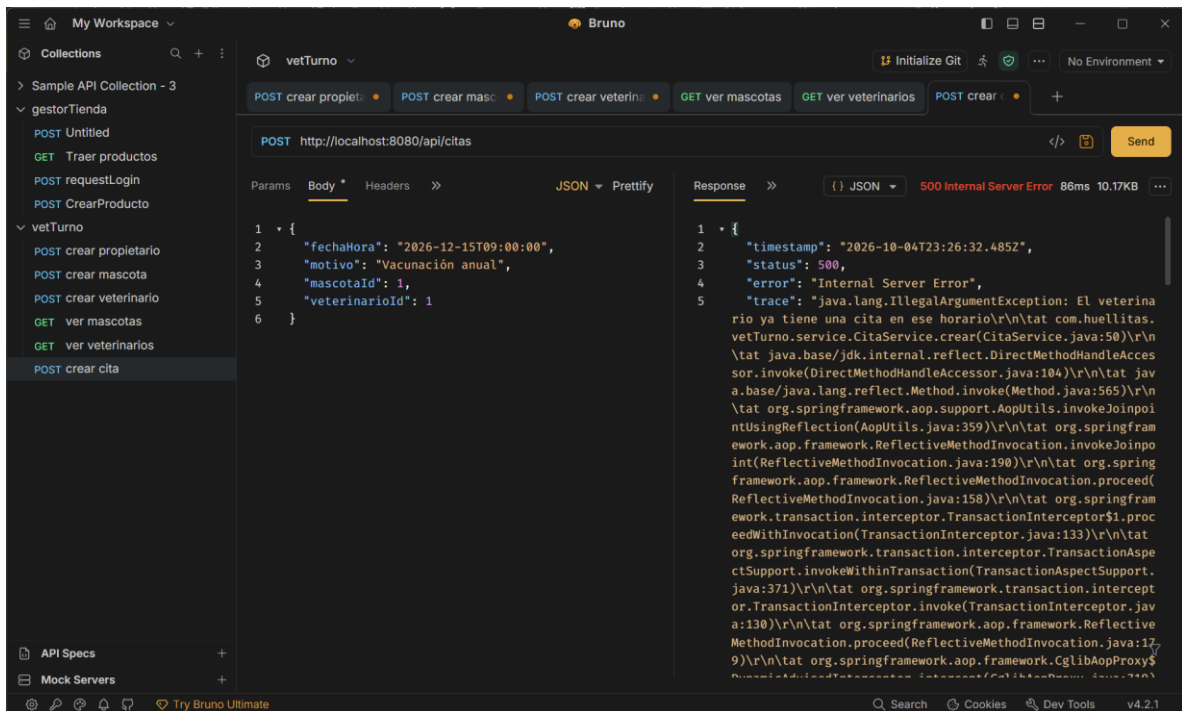
Creación exitosa de una cita con estado 201:



Respuesta controlada ante fecha pasada y horario duplicado:



Consulta de agenda filtrada por veterinario:



PARTE 5:

Hash visible en MySQL, con la contraseña original oculta:

Result Grid			
Filter Rows:			
Export:			
Wrap Cell Content:			
	email	password	rol
▶	paula@huellitas.com	\$2a\$10\$uQV3cmIeko8aIWSSDSih2.whFC92YH...	USER

usuarios 5			
Output			
Action Output			
#	Time	Action	Message
❌ 4	15:02:27	Apply changes to vet_turno	Selected name conflicts with existir
✅ 5	15:02:45	USE vet_turno	0 row(s) affected
✅ 6	17:30:15	SELECT * FROM propietarios LIMIT 0, 1000	1 row(s) returned
✅ 7	17:30:31	SELECT * FROM mascotas LIMIT 0, 1000	1 row(s) returned
✅ 8	17:30:41	SELECT * FROM citas LIMIT 0, 1000	0 row(s) returned
✅ 9	17:30:50	SELECT * FROM veterinarios LIMIT 0, 1000	1 row(s) returned
✅ 10	18:57:26	SELECT email, password, rol FROM usuarios LIMIT 0, 1000	1 row(s) returned

Respuesta de login con el token parcialmente oculto:

The screenshot shows a REST client interface for a project named 'vetTurno'. The selected request is a POST to 'http://localhost:8080/api/auth/register'. The request body is a JSON object with 'email' and 'password' fields. The response is a 200 OK status with a JSON body containing a 'token' field. The token is partially obscured with a grey box.

```
POST http://localhost:8080/api/auth/register
```

Body:

```
{
  "email": "paula@huellitas.com",
  "password": "clave1234"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwYXVsYUBodWVsbG0YXMuy29tIiwicm9sIjoiiVNFUiIsImhhdCI6MTc5MTE1ODIzMiwiZXRhIjo0YXNkxMTYxODMyfQ.2ytszDeuuX0cJEs2gEJX-NaNseTg98_c5SUuIgzxpfa"
}
```

Comparación: POST /api/veterinarios con USER → 403; con ADMIN → 20:

USER:

The screenshot shows a REST client interface for a project named 'vetTurno'. The selected request is a POST to 'http://localhost:8080/api/veterinarios'. The request body is a JSON object with 'nombre' and 'especialidad' fields. The response is a 401 Unauthorized status with an empty body.

```
POST http://localhost:8080/api/veterinarios
```

Body:

```
{
  "nombre": "Dr. Ferrer",
  "especialidad": "Cirugía y Medicina General"
}
```

Response:

```
...
```

ADMIN:

The screenshot shows a REST client interface for a project named 'vetTurno'. The selected request is a POST to 'http://localhost:8080/api/auth/login'. The request body is a JSON object with 'email' and 'password' fields. The response is a 200 OK status with a JSON body containing a 'token' field. The token is partially obscured with a grey box.

```
POST http://localhost:8080/api/auth/login
```

Body:

```
{
  "email": "paula@huellitas.com",
  "password": "clave1234"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwYXVsYUBodWVsbG0YXMuy29tIiwicm9sIjoiiVNFUiIsImhhdCI6MTc5MTE1ODIzMiwiZXRhIjo0YXNkxMTYxODMyfQ.gGySAgVQuPCoxOmvrB41LUn47LbtMpiFZLIiBHANBM"
}
```

7 • `UPDATE usuarios SET rol = 'ADMIN' WHERE email = 'paula@huellitas.com';`

email	password	rol
paula@huellitas.com	\$2a\$10\$uQV3cmIeko8aIWSSDSih2.wHFC92YH...	ADMIN

POST http://localhost:8080/api/veterinarios

Params Body * Headers >> JSON Prettify

```
1 {
2   "nombre": "Dr. Ferrer",
3   "especialidad": "Cirugía y Medicina General"
4 }
```

Response >> {} JSON 201 Created 157ms 75B

```
1 {
2   "id": 2,
3   "nombre": "Dr. Ferrer",
4   "especialidad": "Cirugía y Medicina General"
5 }
```

PARTE 6:

Una respuesta 400 con varios errores de campos:

My Workspace

vetTurno

POST http://localhost:8080/api/auth/register

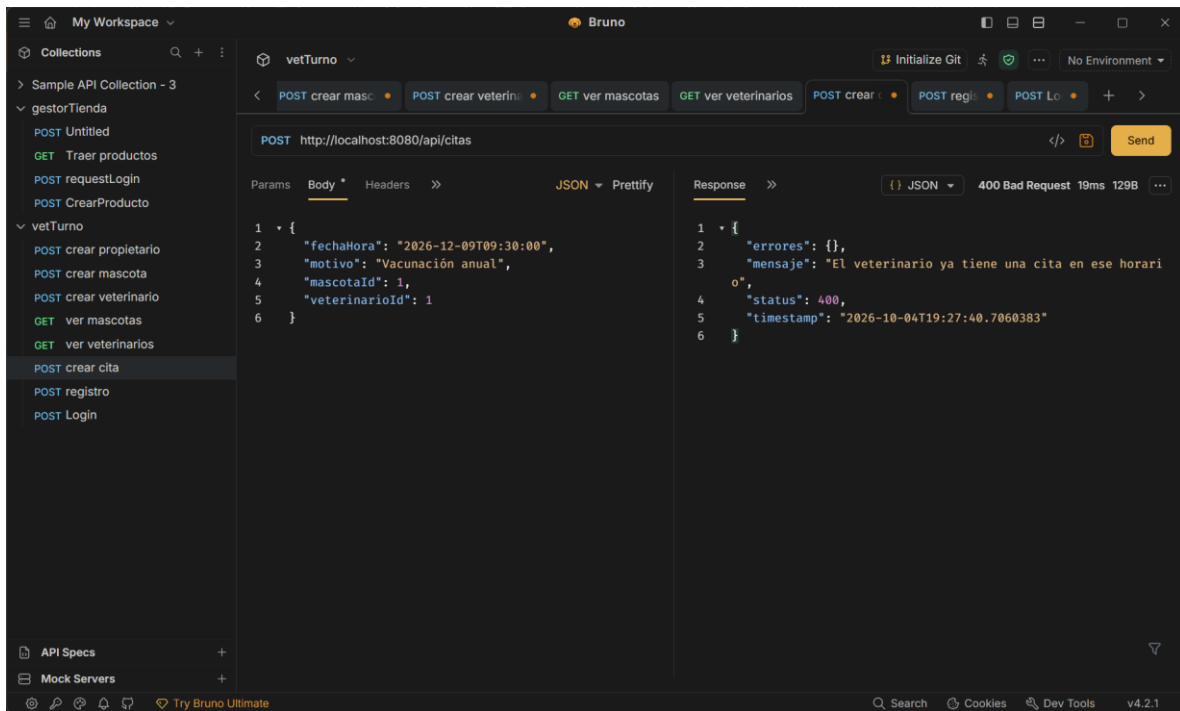
Params Body * Headers >> JSON Prettify

```
1 {
2   "email": "abc",
3   "password": "123"
4 }
```

Response >> {} JSON 400 Bad Request 195ms 225B

```
1 {
2   "errores": {
3     "password": "La contraseña debe tener al menos 8 caracte
4     res",
5     "email": "El email no tiene un formato válido"
6   },
7   "mensaje": "Hay datos inválidos en la solicitud",
8   "status": 400,
9   "timestamp": "2026-10-04T19:21:30.6877347"
}
```

Una respuesta 400 por cruce de horario:



Explicación de por qué el manejador global no reemplaza las reglas de seguridad:

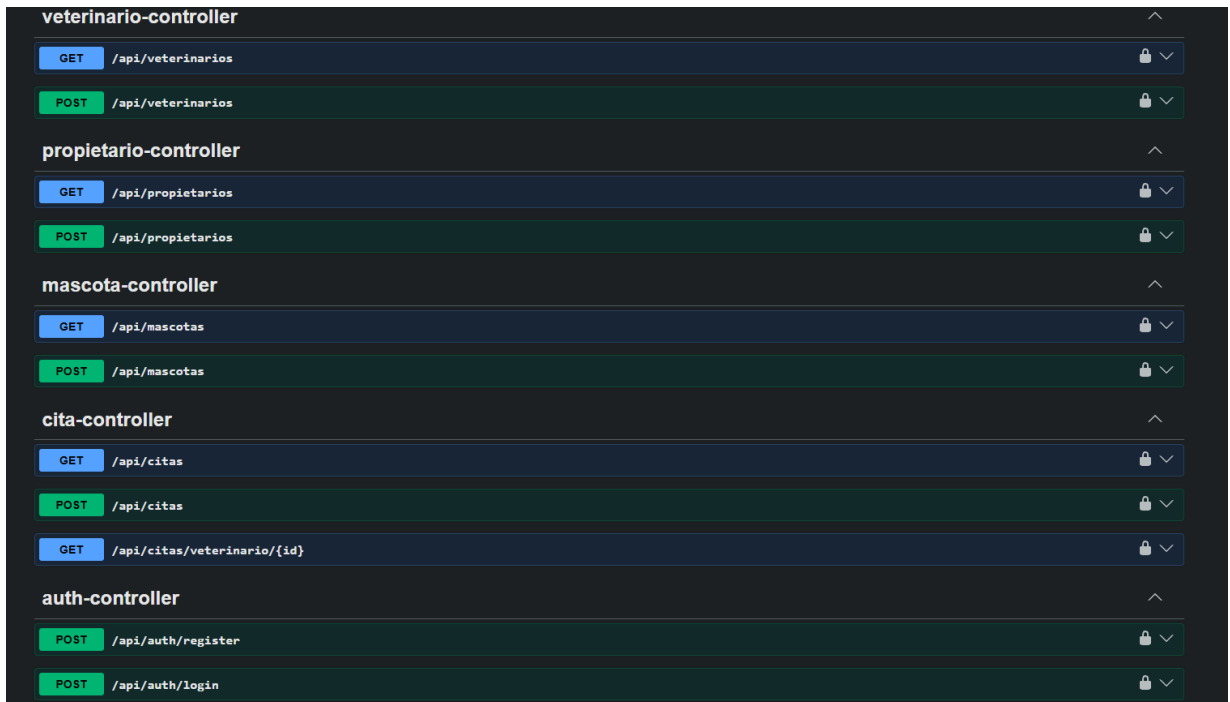
El manejador global solo da formato a los errores que ocurren **dentro** de los controllers (400, 500 con `ApiError`). No decide quién puede entrar. Eso lo hace Spring Security, que actúa **antes**, en la cadena de filtros, y responde 401 o 403 sin que la petición llegue al controller.

Ejemplo: si Paula (USER) intenta `POST /api/veterinarios`, `SecurityConfig` la detiene con un 403 antes de llegar al controller. El manejador nunca se entera, y sin la regla de seguridad nada impediría que creara el veterinario.

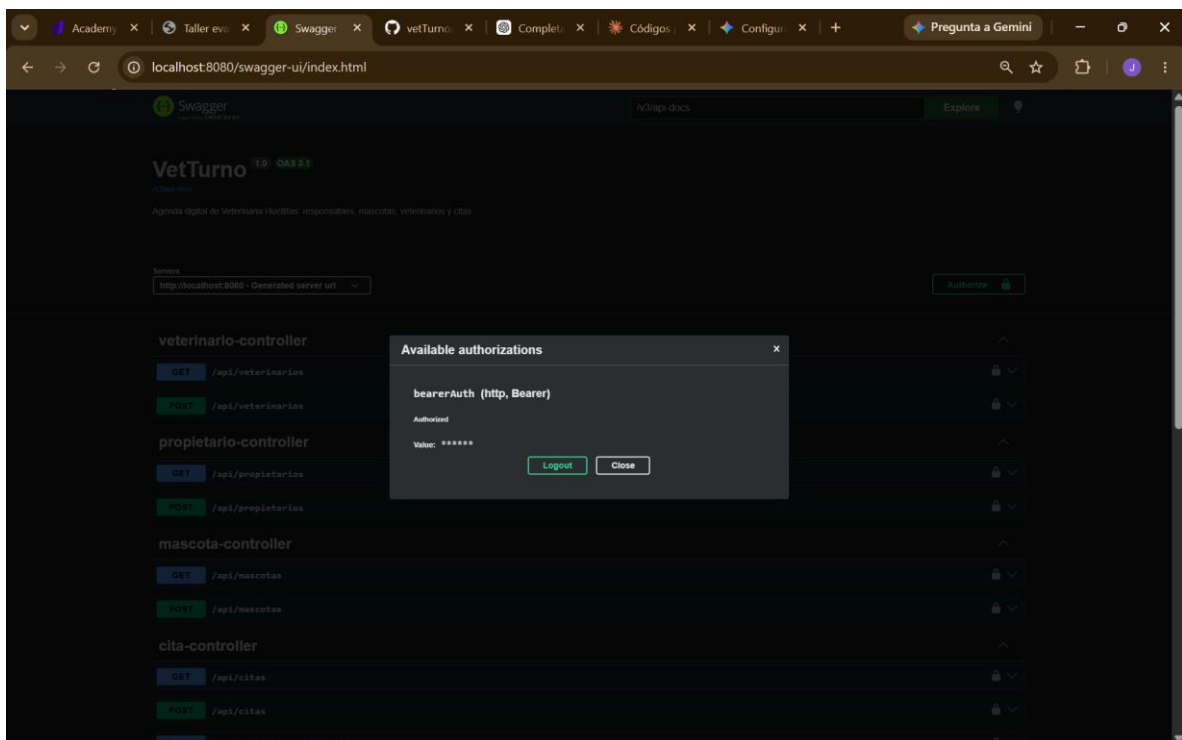
Por eso se complementan: la seguridad decide si la petición entra; el manejador explica el error cuando ocurre. Y por eso `AccessDeniedException` se deja pasar en el handler, para que no se convierta en un 500.

PUNTO 7:

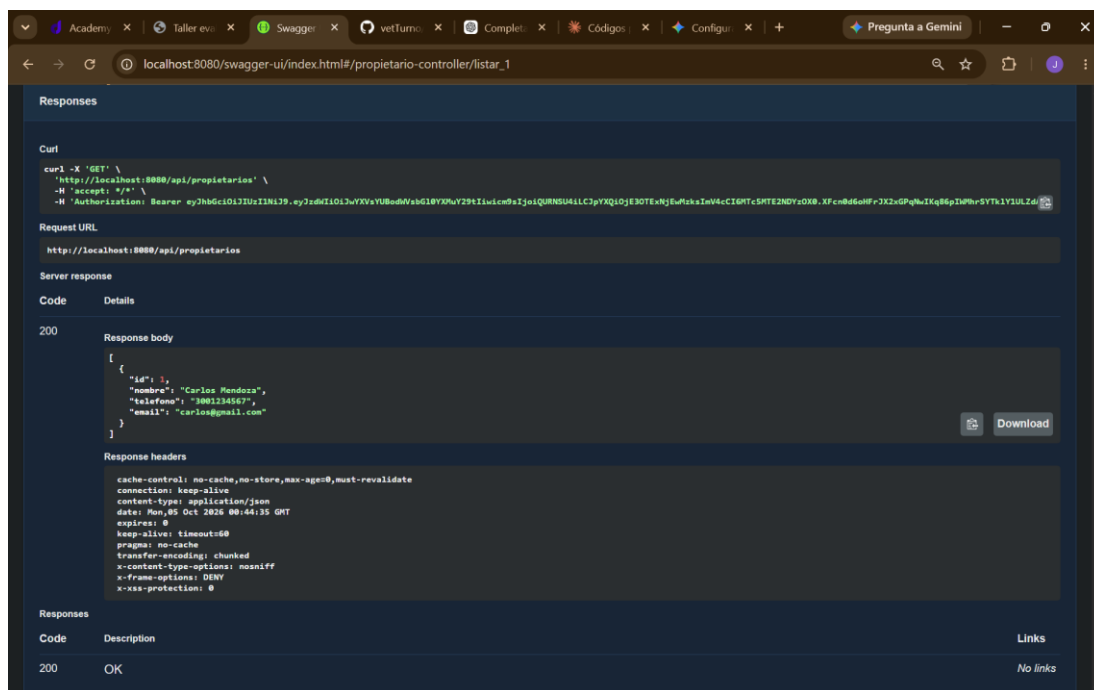
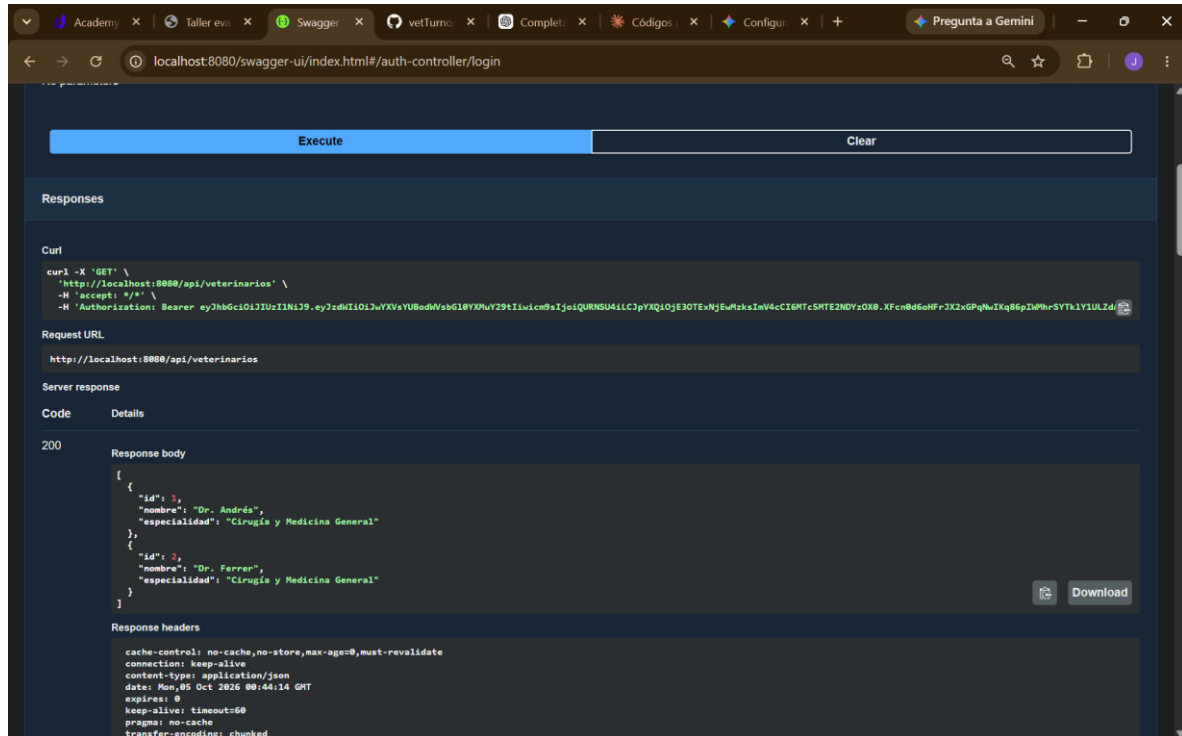
Swagger abre y muestra todos los endpoints obligatorio:



Authorize agrega el token a las peticiones protegidas:



CONFIRMACION DE PERSISTENCIA DE LOS DATOS DESDE SWAGGER



Academy x Taller evi x Swagger x vetTurno x Complet x Códigos x Configu x + Pregunta a Gemini

localhost:8080/swagger-ui/index.html#/mascota-controller/listar_2

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/mascotas' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IHR5cGU6ImF1dG8iLCJpYXQiOiIyMDI0L05E30TExNjEwMks1IiwiaWF0IjE1V4cCI6MTY0OTc5MTE2NDYyOXB9.XFcm0d6oWFr3X2xGPqNwYKq86pDw9hrSYTK1Y1ULZd4'
```

Request URL

http://localhost:8080/api/mascotas

Server response

Code	Details
200	Response body <pre>{ "id": 1, "nombre": "Firulaia", "especie": "Canino", "raza": "Labrador", "propietarioId": 1, "propietarioNombre": "Carlos Mendoza" }</pre> Response headers <pre>cache-control: no-cache, no-store, max-age=0, must-revalidate connection: keep-alive content-type: application/json date: Mon, 05 Oct 2026 00:44:53 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

api

Academy x Taller evi x Swagger x vetTurno x Complet x Códigos x Configu x + Pregunta a Gemini

localhost:8080/swagger-ui/index.html#/cita-controller/listar_3

Request URL

http://localhost:8080/api/citas

Server response

Code	Details
200	Response body <pre>{ "id": 1, "fechaHora": "2026-12-15T09:00:00", "motivo": "Vacunación anual", "mascota": "Firulaia", "propietario": "Carlos Mendoza", "veterinario": "Dr. Andrés" }, { "id": 2, "fechaHora": "2026-12-09T09:30:00", "motivo": "Vacunación anual", "mascota": "Firulaia", "propietario": "Carlos Mendoza", "veterinario": "Dr. Andrés" } }</pre> Response headers <pre>cache-control: no-cache, no-store, max-age=0, must-revalidate connection: keep-alive content-type: application/json date: Mon, 05 Oct 2026 00:45:11 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

api

